

Name: AHREN ABISHEK

Register Number: 192324094

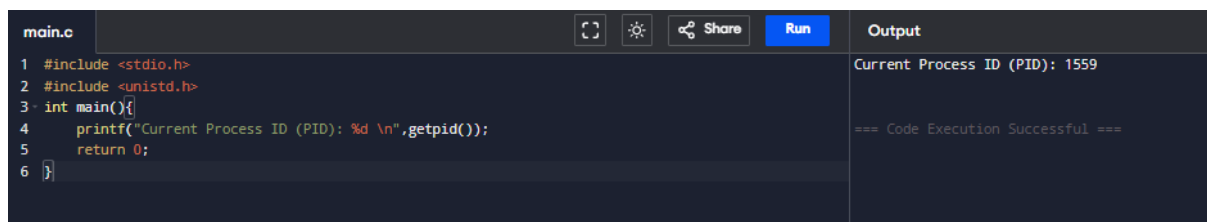
Course Code & Name: CSA0409-Operating Systems

Date: 09-07-2026

1. DISPLAY CURRENT PROCESS ID OF A RUNNING PROGRAM

```
#include <stdio.h>
#include <unistd.h>
int main(){
    printf("Current Process ID (PID): %d \n",getpid());
    return 0;
}
```

OUTPUT:

A screenshot of a code editor interface. The editor has a dark theme. On the left, a file named 'main.c' is open. The code in the editor is:

```
1 #include <stdio.h>
2 #include <unistd.h>
3 int main(){
4     printf("Current Process ID (PID): %d \n",getpid());
5     return 0;
6 }
```

 Above the code, there are icons for a full-screen view, settings, and a 'Share' button. To the right of the code editor is a 'Run' button. To the right of the 'Run' button is a panel titled 'Output'. The 'Output' panel contains the text:

```
Current Process ID (PID): 1559
=== Code Execution Successful ===
```

2. CREATE 2 CHILD PROCESS ID'S AND PRINT THEIR ID'S

```
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
```

```
int main() {
    pid_t pid1, pid2;
```

```
pid1 = fork();

if (pid1 == 0) {
    printf("First Child Process\n");
    printf("Child 1 PID: %d\n", getpid());
} else {
    pid2 = fork();

    if (pid2 == 0) {
        printf("Second Child Process\n");
        printf("Child 2 PID: %d\n", getpid());
    } else {
        printf("Parent Process\n");
        printf("Parent PID: %d\n", getpid());
        printf("First Child PID: %d\n", pid1);
        printf("Second Child PID: %d\n", pid2);
    }
}

return 0;
}
```

OUTPUT:

```
main.c
1 #include <unistd.h>
2 #include <sys/types.h>
3
4
5 int main() {
6     pid_t pid1, pid2;
7
8     pid1 = fork();
9
10    if (pid1 == 0) {
11        printf("First Child Process\n");
12        printf("Child 1 PID: %d\n", getpid());
13    } else {
14        pid2 = fork();
15
16        if (pid2 == 0) {
17            printf("Second Child Process\n");
18            printf("Child 2 PID: %d\n", getpid());
19        } else {
20            printf("Parent Process\n");
21            printf("Parent PID: %d\n", getpid());
22            printf("First Child PID: %d\n", pid1);
23            printf("Second Child PID: %d\n", pid2);
24        }
25    }
26
27    return 0;
28 }
```

Output

```
First Child Process
Parent Process
Child 1 PID: 2064
Parent PID: 2063
First Child PID: 2064
Second Child PID: 2065
Second Child Process
Child 2 PID: 2065

=== Code Execution Successful ===
```

3. SIMULATE A READY QUEUE USING A QUEUE DATA STRUCTURE

```
#include <stdio.h>
```

```
#define MAX 10
```

```
int queue[MAX];
```

```
int front = -1, rear = -1;
```

```
void enqueue(int pid) {
```

```
    if (rear == MAX - 1) {
```

```
        printf("Ready Queue is Full!\n");
```

```
        return;
```

```
    }
```

```
    if (front == -1)
```

```
        front = 0;
```

```
    rear++;
```

```
    queue[rear] = pid;
```

```
    printf("Process %d added to Ready Queue.\n", pid);
```

```
}
```

```

void dequeue() {
    if (front == -1 || front > rear) {
        printf("Ready Queue is Empty!\n");
        return;
    }

    printf("Process %d is executing.\n", queue[front]);
    front++;

    if (front > rear)
        front = rear = -1;
}

void display() {
    int i;

    if (front == -1) {
        printf("Ready Queue is Empty!\n");
        return;
    }

    printf("Ready Queue: ");
    for (i = front; i <= rear; i++) {
        printf("%d ", queue[i]);
    }
    printf("\n");
}

void peek() {
    if (front == -1)
        printf("Ready Queue is Empty!\n");
    else
        printf("Next Process to Execute: %d\n", queue[front]);
}

```

```
}
```

```
int main() {
```

```
    int choice, pid;
```

```
    while (1) {
```

```
        printf("\n--- Ready Queue Simulation ---\n");
```

```
        printf("1. Add Process\n");
```

```
        printf("2. Execute Process\n");
```

```
        printf("3. Display Ready Queue\n");
```

```
        printf("4. Peek Front Process\n");
```

```
        printf("5. Exit\n");
```

```
        printf("Enter your choice: ");
```

```
        scanf("%d", &choice);
```

```
        switch (choice) {
```

```
            case 1:
```

```
                printf("Enter Process ID: ");
```

```
                scanf("%d", &pid);
```

```
                enqueue(pid);
```

```
                break;
```

```
            case 2:
```

```
                dequeue();
```

```
                break;
```

```
            case 3:
```

```
                display();
```

```
                break;
```

```
            case 4:
```

```
                peek();
```

```
                break;
```

case 5:

return 0;

default:

printf("Invalid Choice!\n");

}

}

return 0;

}

OUTPUT:

```
main.c
1 #include <stdio.h>
2 #define MAX 10
3
4 int queue[MAX];
5 int front = -1, rear = -1;
6
7 void enqueue(int pid) {
8     if (rear == MAX - 1) {
9         printf("Ready Queue is Full!\n");
10        return;
11    }
12    if (front == -1)
13        front = 0;
14
15    rear++;
16    queue[rear] = pid;
17    printf("Process %d added to Ready Queue.\n", pid);
18 }
19
20 void dequeue() {
21     if (front == -1 || front > rear) {
22         printf("Ready Queue is Empty!\n");
23         return;
24     }
25
26     printf("Process %d is executing.\n", queue[front]);
27     front++;
28 }
```

--- Ready Queue Simulation ---
1. Add Process
2. Execute Process
3. Display Ready Queue
4. Peek Front Process
5. Exit
Enter your choice: 1
Enter Process ID: 4566
Process 4566 added to Ready Queue.

--- Ready Queue Simulation ---
1. Add Process
2. Execute Process
3. Display Ready Queue
4. Peek Front Process
5. Exit
Enter your choice: 3
Ready Queue: 4566

--- Ready Queue Simulation ---
1. Add Process
2. Execute Process
3. Display Ready Queue
4. Peek Front Process
5. Exit
Enter your choice: |

4. SIMULATE A SIMPLE SCHEDULER THAT EXECUTES PROCESSES ONE BY ONE

```
#include <stdio.h>
```

```
int main() {
```

```
    int n, i, p[10];
```

```
    printf("Enter number of processes: ");
```

```
    scanf("%d", &n);
```

```

for(i = 0; i < n; i++) {
    printf("Enter Process ID: ");
    scanf("%d", &p[i]);
}

printf("\nExecution Order:\n");
for(i = 0; i < n; i++) {
    printf("Executing Process %d\n", p[i]);
}

return 0;
}

```

OUTPUT:

main.c	Output
<pre> 1 #include <stdio.h> 2 3- int main() { 4 int n, i, p[10]; 5 6 printf("Enter number of processes: "); 7 scanf("%d", &n); 8 9- for(i = 0; i < n; i++) { 10 printf("Enter Process ID: "); 11 scanf("%d", &p[i]); 12 } 13 14 printf("\nExecution Order:\n"); 15- for(i = 0; i < n; i++) { 16 printf("Executing Process %d\n", p[i]); 17 } 18 19 return 0; 20 } </pre>	<pre> Enter number of processes: 3 Enter Process ID: 114 Enter Process ID: 113 Enter Process ID: 112 Execution Order: Executing Process 114 Executing Process 113 Executing Process 112 === Code Execution Successful === </pre>

5. READ AND WRITE DATA USING FILE SYSTEM CALLS

```

#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>
#include <string.h>
#include <stdlib.h>

```

```

int main() {
    int fd;
    char data[] = "Hello Operating System";

```

```

char buffer[100];

fd = open("sample.txt", O_CREAT | O_RDWR | O_TRUNC, 0644);

if (fd == -1) {
    perror("Error opening file");
    exit(1);
}

write(fd, data, strlen(data));

lseek(fd, 0, SEEK_SET);

int bytes = read(fd, buffer, sizeof(buffer) - 1);

if (bytes == -1) {
    perror("Error reading file");
    close(fd);
    exit(1);
}

buffer[bytes] = '\0'; // Null terminate the string

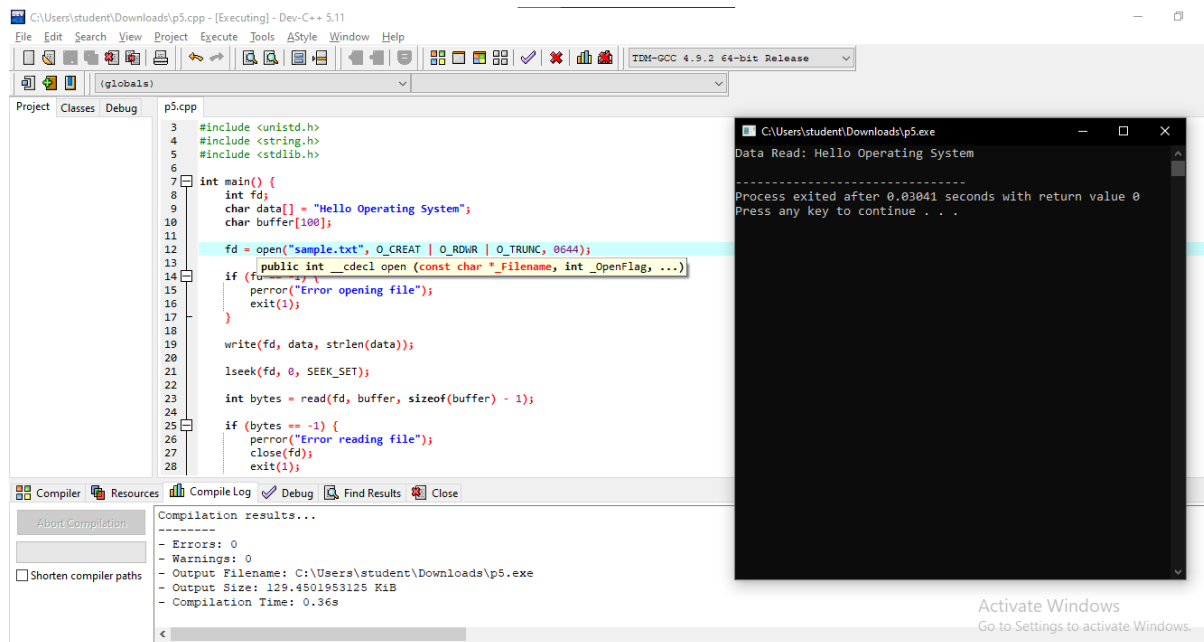
printf("Data Read: %s\n", buffer);

close(fd);

return 0;
}

```

OUTPUT:



6. SIMULATE PAGE REPLACEMENT USING A LIST OR QUEUE

```
#include <stdio.h>
```

```
int main() {
```

```
    int frames[3], pages[] = {1,2,3,4,1,2,5};
```

```
    int i, j, front = 0, faults = 0, found;
```

```
    for(i=0;i<3;i++)
```

```
        frames[i] = -1;
```

```
    for(i=0;i<7;i++) {
```

```
        found = 0;
```

```
        for(j=0;j<3;j++) {
```

```
            if(frames[j] == pages[i]) {
```

```
                found = 1;
```

```
                break;
```

```
            }
```

```
        }
```

```
    }
```

```

        if(!found) {
            frames[front] = pages[i];
            front = (front + 1) % 3;
            faults++;
        }
    }

    printf("Page Faults = %d\n", faults);

    return 0;
}

```

OUTPUT:

The screenshot shows a C code editor with a file named `main.c`. The code implements a page replacement algorithm. It initializes an array `frames` of size 3 with values `-1`, and an array `pages` with values `{1, 2, 3, 4, 1, 2, 5}`. It then iterates through the `pages` array. For each page, it checks if it is already in the `frames` array. If not, it replaces the page in the `frames` array at the `front` index and increments the `faults` counter. The `front` index is updated to `(front + 1) % 3`. Finally, it prints the total number of page faults, which is 7.

```

1  #include <stdio.h>
2
3  int main() {
4      int frames[3], pages[] = {1,2,3,4,1,2,5};
5      int i, j, front = 0, faults = 0, found;
6
7      for(i=0;i<3;i++)
8          frames[i] = -1;
9
10     for(i=0;i<7;i++) {
11         found = 0;
12
13         for(j=0;j<3;j++) {
14             if(frames[j] == pages[i]) {
15                 found = 1;
16                 break;
17             }
18         }
19
20         if(!found) {
21             frames[front] = pages[i];
22             front = (front + 1) % 3;
23             faults++;
24         }
25     }
26
27     printf("Page Faults = %d\n", faults);
28 }

```

The output of the program is displayed on the right side of the editor:

```

Page Faults = 7
=== Code Execution Successful ===

```

7. ALLOCATE MEMORY BLOCKS USING ARRAYS

```
#include <stdio.h>
```

```

int main() {
    int block[] = {100, 500, 200, 300};

```

```

int process[] = {212, 417, 112};

int i, j;

for(i=0;i<3;i++) {
    for(j=0;j<4;j++) {
        if(block[j] >= process[i]) {
            printf("Process %d allocated to Block %d\n", process[i], j+1);
            block[j] -= process[i];
            break;
        }
    }

    if(j==4)
        printf("Process %d Not Allocated\n", process[i]);
}

return 0;
}

```

OUTPUT:

main.c	Output
<pre> 1 #include <stdio.h> 2 3 int main() { 4 int block[] = {100, 500, 200, 300}; 5 int process[] = {212, 417, 112}; 6 int i, j; 7 8 for(i=0;i<3;i++) { 9 for(j=0;j<4;j++) { 10 if(block[j] >= process[i]) { 11 printf("Process %d allocated to Block %d\n", process[i], j+1); 12 block[j] -= process[i]; 13 break; 14 } 15 } 16 17 if(j==4) 18 printf("Process %d Not Allocated\n", process[i]); 19 } 20 21 return 0; 22 } </pre>	<pre> Process 212 allocated to Block 2 Process 417 Not Allocated Process 112 allocated to Block 2 === Code Execution Successful === </pre>

8. CREATE 2 THREADS THAT PRINT NUMBERS ALTERNATELY

```
#include <stdio.h>
#include <pthread.h>

void *even(void *arg) {
    int i;
    for(i=2;i<=10;i+=2)
        printf("Even: %d\n", i);
    return NULL;
}

void *odd(void *arg) {
    int i;
    for(i=1;i<=9;i+=2)
        printf("Odd : %d\n", i);
    return NULL;
}

int main() {
    pthread_t t1, t2;

    pthread_create(&t1, NULL, odd, NULL);
    pthread_create(&t2, NULL, even, NULL);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    return 0;
}
```

OUTPUT:

```
main.c
1 #include <stdio.h>
2 #include <pthread.h>
3
4 void *even(void *arg) {
5     int i;
6     for(i=2;i<=10;i+=2)
7         printf("Even: %d\n", i);
8     return NULL;
9 }
10
11 void *odd(void *arg) {
12     int i;
13     for(i=1;i<=9;i+=2)
14         printf("Odd : %d\n", i);
15     return NULL;
16 }
17
18 int main() {
19     pthread_t t1, t2;
20
21     pthread_create(&t1, NULL, odd, NULL);
22     pthread_create(&t2, NULL, even, NULL);
23
24     pthread_join(t1, NULL);
25     pthread_join(t2, NULL);
26
27     return 0;
28 }
```

Output

```
Odd : 1
Odd : 3
Odd : 5
Odd : 7
Odd : 9
Even: 2
Even: 4
Even: 6
Even: 8
Even: 10

=== Code Execution Successful ===
```

9. IMPLEMENT A BOUNDED BUFFER USING A QUEUE

```
#include <stdio.h>
```

```
#define SIZE 5
```

```
int buffer[SIZE];
```

```
int front = -1, rear = -1;
```

```
void produce(int item) {
    if(rear == SIZE-1) {
        printf("Buffer Full\n");
        return;
    }
```

```
    if(front == -1)
        front = 0;
```

```
    buffer[++rear] = item;
}
```

```
void consume() {
    if(front == -1 || front > rear) {
        printf("Buffer Empty\n");
        return;
    }

    printf("Consumed: %d\n", buffer[front++]);

    if(front > rear)
        front = rear = -1;
}

int main() {
    produce(10);
    produce(20);
    produce(30);

    consume();
    consume();

    return 0;
}
```

OUTPUT:

```
main.c
14  if(front == -1)
15      front = 0;
16
17  buffer[++rear] = item;
18  }
19
20  void consume() {
21      if(front == -1 || front > rear) {
22          printf("Buffer Empty\n");
23          return;
24      }
25
26      printf("Consumed: %d\n", buffer[front++]);
27
28      if(front > rear)
29          front = rear = -1;
30  }
31
32  int main() {
33      produce(10);
34      produce(20);
35      produce(30);
36
37      consume();
38      consume();
39
40      return 0;
41  }
```

Output

```
Consumed: 10
Consumed: 20

=== Code Execution Successful ===
```

10. CALCULATE AVERAGE WAITING TIME FOR A GIVEN LIST OF PROCESSES

```
#include <stdio.h>
```

```
int main() {
```

```
    int n, bt[10], wt[10];
```

```
    int i;
```

```
    float avg = 0;
```

```
    printf("Enter number of processes: ");
```

```
    scanf("%d", &n);
```

```
    for(i=0;i<n;i++) {
```

```
        printf("Burst Time P%d: ", i+1);
```

```
        scanf("%d", &bt[i]);
```

```
    }
```

```
    wt[0] = 0;
```

```
    for(i=1;i<n;i++)
```

```
wt[i] = wt[i-1] + bt[i-1];
```

```
printf("\nProcess\tBurst\tWaiting\n");
```

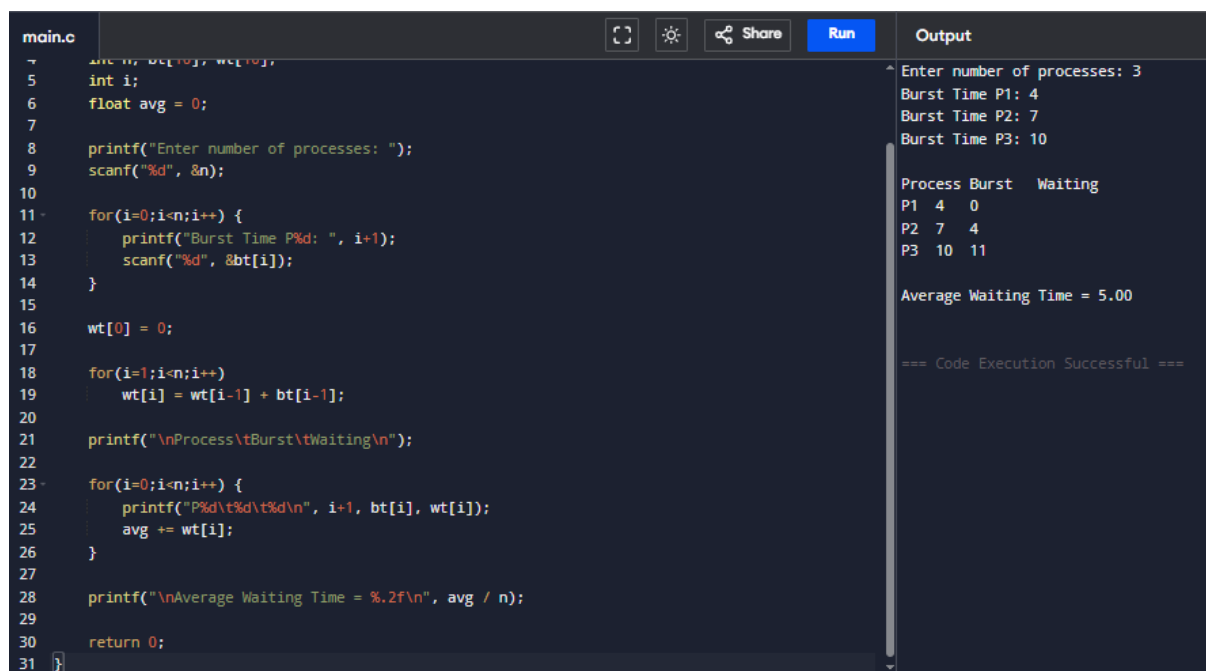
```
for(i=0;i<n;i++) {  
    printf("P%d\t%d\t%d\n", i+1, bt[i], wt[i]);  
    avg += wt[i];  
}
```

```
printf("\nAverage Waiting Time = %.2f\n", avg / n);
```

```
return 0;
```

```
}
```

OUTPUT:



The screenshot shows a code editor with a C program and its output. The code calculates the average waiting time for three processes based on their burst times. The output displays the input burst times, a table of process details, and the final average waiting time.

```
main.c  
1  int n, bt[10], wt[10];  
2  int i;  
3  float avg = 0;  
4  
5  printf("Enter number of processes: ");  
6  scanf("%d", &n);  
7  
8  for(i=0;i<n;i++) {  
9      printf("Burst Time P%d: ", i+1);  
10     scanf("%d", &bt[i]);  
11 }  
12  
13 wt[0] = 0;  
14  
15 for(i=1;i<n;i++)  
16     wt[i] = wt[i-1] + bt[i-1];  
17  
18 printf("\nProcess\tBurst\tWaiting\n");  
19  
20 for(i=0;i<n;i++) {  
21     printf("P%d\t%d\t%d\n", i+1, bt[i], wt[i]);  
22     avg += wt[i];  
23 }  
24  
25 printf("\nAverage Waiting Time = %.2f\n", avg / n);  
26  
27 return 0;
```

Output:

```
Enter number of processes: 3  
Burst Time P1: 4  
Burst Time P2: 7  
Burst Time P3: 10  
  
Process Burst    Waiting  
P1  4  0  
P2  7  4  
P3 10 11  
  
Average Waiting Time = 5.00  
  
=== Code Execution Successful ===
```